

Expression of the Dimensional Ladder as Dual-Framed Description According to the Lambda Principle of Irreducibility

Established arithmetic, reciprocal necessity, projective frame edges, identity transport, and a Lean-verified core

Lu Semita · EmergenceByDesign

9 September 2026 · V2 with integrated verification receipt

Abstract

This note organizes familiar mathematics into a single representational sequence called the **Dimensional Ladder**. It does not propose new arithmetic, new projective geometry, or a new definition of identity. The constructions are standard: signed addition around an origin; natural-number multiplication as repeated addition and exact Cartesian-product enumeration; reciprocal inversion as the unique multiplicative inverse required to preserve the field laws for nonzero elements; projective completion of an affine line by a point at infinity; reciprocal coordinate charts whose transition is $x \mapsto x^{-1}$; and observation fibres that make higher-state variation invisible to a lower registry until an additional independent relation is supplied.

The organizing claim is equally modest. Distinct mathematical objects remain distinct by definition. Their identities remain literal identities inside their declared structures. What can be compared across rungs is the **transport of relational content**: what remains invariant, what is transformed, what becomes newly expressible, and what is lost under projection. The label **Lambda Principle of Irreducibility** names that recurring descriptive structure; the label is not a premise required for any theorem in this paper.

The Lean 4 source included in Appendix C formalizes the elementary algebraic, Cartesian-product, reciprocal, projective-pair, identity-transport, and fibre claims. The formal layer is intentionally narrower than the prose. A proof assistant checks the proposition encoded in its formal environment; it does not choose the intended meaning of an informal term or certify an interpretation that was never formalized.

Verification result, 9 September 2026. Lean Web accepted the corrected 30-theorem core and the `safeDiv` definition under Lean 4.33.0, with 0 errors and 0 warnings. Appendices A–E include the receipt, axiom ledger, complete submitted source, captured output, and source-identity record.

0. Standing: what is and is not being claimed

The paper uses **dimension** in a restricted constructive sense: the number of independent coordinates required by a declared Cartesian enumeration or state description. This does not imply that every arithmetic operation changes topological dimension.

The paper uses **frame** for a declared representational structure: its domain, coordinates, admissible operations, and interpretation. A **frame edge** is where a particular coordinate description ceases to assign finite values. The completed object need not have a geometric boundary there.

Standard arithmetic includes 0 as a number. The Dimensional Ladder additionally uses 0 as the common additive origin or null reference of the signed grid. This additional descriptive role does not alter any arithmetic law.

Plain statement. The equations come first. The paper places them in one continuous diagram and gives the recurrent diagram a name. A reader may reject the name *Lambda* without changing any equation below.

1. The simplest demonstration

Start with a symmetric integer grid:

$$\cdots, -3, -2, -1, 0, 1, 2, 3, \cdots.$$

Zero is the common origin. At the origin there is no displacement from the origin:

$$0 - 0 = 0.$$

Addition and subtraction are signed translations on this one-dimensional registry. Subtraction is not a second unrelated primitive:

$$a - b = a + (-b).$$

The two rays are balanced by additive inversion:

$$a \longleftrightarrow -a, \quad -(-a) = a.$$

Now repeat a one-dimensional additive pattern across a second independent index. Three copies of a four-unit row give a 3×4 grid:

$$\begin{array}{cccc} \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \end{array}$$

There are exactly twelve cells:

$$|[3] \times [4]| = 12.$$

The grid is therefore not a false picture of natural-number multiplication. It is an exact Cartesian-product representation of it.

Division by a nonzero factor changes the multiplicative reference. If four old units are declared to constitute one new unit, then

$$1 \text{ old unit} = \frac{1}{4} \text{ new unit},$$

and therefore

$$12 \text{ old units} = 12 \cdot \frac{1}{4} = 3 \text{ new units}.$$

The reciprocal is not a sign change. It reverses which member of a multiplicative comparison serves as reference:

$$\frac{x}{y} \longleftrightarrow \frac{y}{x}.$$

Projectively, this reciprocal exchange becomes exact at the affine frame edge:

$$[X : Z] \longleftrightarrow [Z : X],$$

so that

$$0 = [0 : 1] \longleftrightarrow [1 : 0] = \infty.$$

The first grid did not fail. It reached a state it cannot express with a finite value in its own coordinate. The reciprocal chart continues the same completed relation from the opposite description.

That is the entire ladder in miniature:

$$\begin{aligned} &\text{origin} \rightarrow \text{directed enumeration} \rightarrow \text{independent enumeration} \\ &\quad \rightarrow \text{reciprocal re-reference} \rightarrow \text{dual frame} \rightarrow \text{larger atlas.} \end{aligned}$$

The remaining sections state this simple picture rigorously.

2. Zero: additive identity, multiplicative annihilator, projective boundary marker

Within ordinary arithmetic,

$$a + 0 = a, \quad a - 0 = a.$$

Thus the additive registry treats 0 as identity for translation. No nontrivial directed displacement is generated by the pair (0, 0).

Multiplication gives the same element a different algebraic role:

$$a \cdot 0 = 0.$$

The multiplication map

$$M_0(a) = a \cdot 0$$

is constant. Every input lies in the same fibre over zero. In this exact sense multiplication by zero erases every multiplicative distinction represented only by the input value.

Ordinary field division, however, requires a nonzero denominator. There is no field element c satisfying

$$0 \cdot c = 1.$$

Indeed $0 \cdot c = 0$ for every c . Hence zero has no multiplicative inverse in a nontrivial field.

This is the first important separation of descriptive roles:

- additive identity: $a + 0 = a$;
- multiplicative annihilator: $a0 = 0$;
- no field reciprocal: 0^{-1} is not admitted by the ordinary partial division operation;

- projective frame marker: homogeneous pairs $[a : 0]$ with $a \neq 0$ represent the projective point at infinity.

The same written symbol participates in different relations because the surrounding operation has changed. No exotic interpretation is required; the definitions already distinguish the roles.

3. The same calling law: addition lifted into independent enumeration

For natural numbers, multiplication is generated recursively from addition:

$$a \cdot 0 = 0,$$

$$a \cdot (n + 1) = a \cdot n + a.$$

Equivalently,

$$ab = \underbrace{b + \dots + b}_{a \text{ copies}}.$$

Let

$$[a] = \{1, \dots, a\}, \quad [b] = \{1, \dots, b\}.$$

Then

$$|[a] \times [b]| = ab.$$

Moreover,

$$[a] \times [b] = \bigsqcup_{i \in [a]} (\{i\} \times [b]).$$

Each fibre contains exactly b elements, so

$$|[a] \times [b]| = \sum_{i \in [a]} b = ab.$$

This gives a precise meaning to the claim that addition and multiplication express the **same calling law at successive enumeration ranks**:

Multiplication is not literally the addition operator. Natural-number multiplication repeats an additive unit-pattern once for every value of a new independent index.

Define

$$C_1(a_1) = a_1,$$

and recursively

$$C_{k+1}(a_1, \dots, a_{k+1}) = \sum_{j=1}^{a_{k+1}} C_k(a_1, \dots, a_k).$$

Then

$$C_{k+1} = C_k a_{k+1},$$

and therefore

$$C_k(a_1, \dots, a_k) = \prod_{i=1}^k a_i.$$

For equal side lengths,

$$C_k(a, \dots, a) = a^k.$$

Thus the discrete ladder is the exact Cartesian sequence

$$[a] \rightarrow [a]^2 \rightarrow [a]^3 \rightarrow \dots \rightarrow [a]^k.$$

Each rung takes the already constructed pattern as one unit and calls it across an additional independent index.

4. Why the reciprocal is not a convenient afterthought

Let F be a field and let $b \neq 0$. By the field axioms, b has a multiplicative inverse b^{-1} satisfying

$$bb^{-1} = 1.$$

That inverse is unique. If another element c also satisfies

$$bc = 1,$$

then

$$c = 1c = (b^{-1}b)c = b^{-1}(bc) = b^{-1}.$$

Therefore once the multiplicative identity and inverse laws have been fixed, the operation that undoes multiplication by a nonzero b is forced:

$$a/b = ab^{-1}.$$

The reciprocal is consequently not a mnemonic convention grafted onto otherwise unrelated rules. It is the unique algebraic element that makes multiplicative reversal preserve the identity:

$$b \cdot b^{-1} = 1.$$

The statement remains relative to the declared field structure. One may choose a different algebraic structure, but **inside a field** the inverse relation is not arbitrary.

Write the directed comparison

$$R(x \mid y) = \frac{x}{y}, \quad y \neq 0.$$

Exchanging reference and measured term gives

$$R(y \mid x) = \frac{y}{x} = R(x \mid y)^{-1}, \quad x, y \neq 0.$$

If $\mathcal{F}$ denotes this exchange, then

$$\mathcal{F}^2 = \text{id}.$$

This is the **reciprocal frame flip** used in the Dimensional Ladder. It must not be confused with additive sign reflection $x \mapsto -x$.

5. Division as descent through a known fibre

Let

$$\pi_A : A \times B \rightarrow A$$

be projection onto the first factor of two finite sets. Every point $a \in A$ has a fibre

$$\pi_A^{-1}(a) = \{a\} \times B$$

of cardinality $|B|$. Hence

$$|A \times B| = |A||B|,$$

and if $|B| \neq 0$,

$$|A| = \frac{|A \times B|}{|B|}.$$

This is an exact finite model of dimensional descent. The higher-rank Cartesian object is projected onto one generating factor, and division removes the known multiplicity of the forgotten fibre.

The ascent and descent are reversible when the missing relation is supplied, but they are not procedurally identical. Ascent adds an independent index; descent must know the factor or fibre that is being removed.

6. Projective completion: the exact frame-edge flip

A point of the real projective line $\mathbb{RP}^1$ is represented by homogeneous coordinates

$$[X : Z],$$

where

$$[X : Z] = [\lambda X : \lambda Z] \quad (\lambda \neq 0),$$

and $[0 : 0]$ is excluded.

The affine chart $Z \neq 0$ uses

$$x = \frac{X}{Z}.$$

Its missing projective point is

$$\infty = [1 : 0].$$

For every $a \neq 0$,

$$[a : 0] = [1 : 0].$$

This does **not** redefine field division by zero. It states a projective equivalence between homogeneous coordinate pairs.

The reciprocal chart $X \neq 0$ uses

$$u = \frac{Z}{X}.$$

Where both charts apply,

$$u = \frac{1}{x}.$$

Now exchange the homogeneous coordinates:

$$\mathcal{F}([X : Z]) = [Z : X].$$

Then

$$\mathcal{F}^2 = \text{id},$$

and

$$\mathcal{F}([0 : 1]) = [1 : 0], \quad \mathcal{F}([1 : 0]) = [0 : 1].$$

Therefore

$$0 \longleftrightarrow \infty$$

is an exact projective frame exchange.

Topologically,

$$\mathbb{RP}^1 \cong S^1.$$

The completed relation is closed even though no single affine real coordinate assigns finite values everywhere. The apparent edge belongs to the chart, not to a break in the projective object.

Projectivization also identifies common nonzero scale:

$$[X : Z] = [\lambda X : \lambda Z].$$

Thus absolute homogeneous magnitude is removed while relational position survives. This is the precise mathematical sense in which the wrapped description can preserve its internal law while no single affine grid carries the whole closed relation in finite coordinates.

7. Invisible dimensions as observation fibres

The same mathematics appears in perspective.

For an ideal pinhole camera, a three-dimensional point (X, Y, Z) , with $Z \neq 0$, projects to

$$\pi(X, Y, Z) = \left(f \frac{X}{Z}, f \frac{Y}{Z} \right).$$

For any nonzero scalar λ ,

$$\pi(\lambda X, \lambda Y, \lambda Z) = \pi(X, Y, Z).$$

A point can therefore move along a viewing ray while producing no image-plane displacement. The radial degree of freedom lies inside one fibre of the projection.

More generally, let

$$\Phi : X \rightarrow Y$$

be an observation map. If

$$x_1 \neq x_2, \quad \Phi(x_1) = \Phi(x_2),$$

then every deterministic postprocessing $g : Y \rightarrow Z$ satisfies

$$g(\Phi(x_1)) = g(\Phi(x_2)).$$

No amount of reprocessing the same observation can split a distinction that the observation map already identified.

A second displaced camera, motion parallax, stereo disparity, an orientation marker, or another independent measurement enlarges the registry and may split the fibre. Depth perception by triangulation is the elementary example.

The same principle applies to rotation. If a state has position and orientation,

$$s = (p, R),$$

but the registry records only

$$\Pi(p, R) = p,$$

then

$$\Pi(p, R_1) = \Pi(p, R_2)$$

for every pair of orientations over the same position. A symmetric unmarked point can rotate without positional displacement. Orientation becomes visible only when the description includes a marker or another relation capable of distinguishing it.

This is the exact ladder law:

Variation that remains inside a fibre of the current registry is invisible to that registry. Dimensional enlargement occurs when another independent relation distinguishes states that the lower representation necessarily identifies.

Astra 15A expresses the same scope discipline operationally: a target factors through an observation exactly when it is constant on every fibre of that observation, and a record does not manufacture a verdict outside its declared scope.

8. Literal identity and transport between rungs

Identity is not weakened anywhere in this construction.

Let a frame R have a language L_R , a domain D_R , and an interpretation

$$\llbracket \cdot \rrbracket_R : L_R \rightarrow D_R.$$

Inside that frame,

$$s = t$$

means literally

$$\llbracket s \rrbracket_R = \llbracket t \rrbracket_R.$$

Now let R' be another frame. To transport a term from R to R' , specify a syntactic translation

$$\tau : L_R \rightarrow L_{R'}$$

and a carrier map

$$F : D_R \rightarrow D_{R'}.$$

A structure-preserving bridge satisfies

$$F(\llbracket t \rrbracket_R) = \llbracket \tau(t) \rrbracket_{R'}$$

for every transported term t .

Then literal identity transports:

$$\llbracket s \rrbracket_R = \llbracket t \rrbracket_R$$

implies

$$\llbracket \tau(s) \rrbracket_{R'} = \llbracket \tau(t) \rrbracket_{R'}.$$

This is simply preservation of equality by a function together with a commuting translation diagram.

Without such a bridge, the same printed symbol in two frames does not by itself establish continuity of meaning.

Transport principle. Identity is exact within a declared frame. Carrying identity across a frame boundary requires a map specifying what is preserved, what is transformed, and what new distinctions the successor frame admits.

This principle does not invent a new notion of identity. It states the ordinary mathematical burden already present whenever one uses embeddings, quotients, projections, change of basis, coordinate changes, representations, or functors.

9. Established identities seen in one relational context

The ladder does not identify distinct mathematical objects with one another. It compares how exact identity is expressed at different representational rungs.

9.1 Sophie Germain: universal polynomial identity

In any commutative ring,

$$a^4 + 4b^4 = (a^2 - 2ab + 2b^2)(a^2 + 2ab + 2b^2).$$

The two sides are literally the same polynomial element after expansion. The paired quadratic factors are exchanged by $b \mapsto -b$, while their product remains unchanged.

This is a universal identity: it holds for every admissible substitution in the declared ring.

9.2 Euler: exact analytic closure at a distinguished value

Euler's formula gives

$$e^{i\theta} = \cos \theta + i \sin \theta.$$

At $\theta = \pi$,

$$e^{i\pi} = -1,$$

and therefore

$$e^{i\pi} + 1 = 0.$$

This is an exact evaluated relation in the complex-analytic frame, not a polynomial identity under arbitrary substitution. The enlargement matters: complex exponentiation represents angular phase and multiplicative structure in one analytic operation.

9.3 Noether: variational identities and conserved relations

Noether's first theorem relates continuous symmetries of an action to conserved currents on solutions of the Euler-Lagrange equations. Noether's second theorem yields differential identities among Euler-Lagrange expressions when the action has local or gauge symmetries.

The objects have changed from numbers and polynomials to fields, functionals, derivatives, and symmetry actions. The exact identity or invariance relation has not become metaphorical; its formal expression has changed with the frame.

The common architecture is therefore:

defined objects + defined transformation + preserved relation.

What changes across the ladder is the representational class capable of carrying the relation.

The claim is not

Sophie Germain = Euler = Noether.

The claim is that separately defined structures can be considered in one relational context without erasing their definitions because the relevant invariants and transformation laws are explicit.

10. The pharmakon of familiar operations

A mature notation is useful partly because it suppresses work that users no longer need to reconstruct each time. That efficiency has a cost: a rule can become so familiar that its structural necessity disappears from view and it begins to look like an arbitrary instruction inherited from a textbook.

The reciprocal provides a clean example.

The classroom rule

$$\frac{a}{b} = a \cdot \frac{1}{b}$$

can be memorized as a convention. But inside a field the inverse b^{-1} is uniquely forced by

$$bb^{-1} = 1.$$

Likewise, the prohibition on ordinary division by zero follows because no element can satisfy

$$0c = 1.$$

The projective point $[1 : 0]$ does not repeal that rule; it belongs to a different representational construction with a different definition of admissible coordinate pairs.

The **pharmakon** here is therefore descriptive rather than mystical: notation compresses a successful structural relation so efficiently that the reason for the relation can become invisible through use.

The Dimensional Ladder reverses that compression long enough to display the dependency:

primitive laws $\rightarrow$ operation $\rightarrow$ inverse
 $\rightarrow$ frame transition $\rightarrow$ preserved identity.

Nothing new has been inserted into the mathematics. The familiar operations are being unfolded so their relations can be inspected together.

11. Lambda Principle of Irreducibility as a registry name

The name **Lambda Principle of Irreducibility** is used here for the recurring descriptive pattern:

A representation can remain exact on its declared domain while identifying or omitting distinctions required by a larger target relation. Operations internal to that representation cannot recover a distinction already collapsed by the representation map. Resolution requires an enlarged coordinate, reciprocal frame, structural theorem, or larger atlas.

The mathematical results do not depend on this name.

A reader may replace *Lambda* with any other label while retaining all of the following:

1. $a - b = a + (-b)$.
2. $a(n + 1) = an + a$ for natural-number multiplication.
3. $|A \times B| = |A||B|$ for finite sets.
4. Nonzero multiplicative inverses are unique in a field.
5. $a/b = ab^{-1}$ for $b \neq 0$.
6. $[X : Z] \mapsto [Z : X]$ is an involutive homogeneous-coordinate exchange.
7. It exchanges $[0 : 1]$ and $[1 : 0]$.
8. The reciprocal affine transition is $u = 1/x$ on chart overlap.
9. Common nonzero homogeneous scaling does not change a projective point.
10. Deterministic postprocessing cannot split an observation fibre.
11. Perspective projection identifies scalar multiples on one viewing ray.
12. Equality transports through a declared structure-preserving map.
13. Polynomial, analytic, and variational identities retain their own exact local definitions even when compared in a common relational registry.

The Lambda contribution is the organizational claim that these established structures exhibit a recurrent problem of frame-local completeness and transport.

12. Lean verification and the non-oracle boundary

The Lean source printed in Appendix C formalizes the narrow mathematical kernel. The completed Lean Web run accepted all 30 theorem declarations and the `safeDiv` definition. Appendix A records the environment and result; Appendix B maps every audited declaration to its axiom report; Appendices C and D supply the submitted source and captured output.

Its theorem set includes:

- additive identity, subtraction as additive inverse, and multiplicative annihilation by zero;
- recursive natural-number multiplication;
- cardinality of finite Cartesian grids;
- reciprocal involution and uniqueness of the inverse that restores multiplicative identity;
- a gated `safeDiv` requiring an explicit proof that the denominator is nonzero;
- homogeneous coordinate exchange, origin/infinity exchange, and projective cross-relation invariance;
- reciprocal affine charts and common-scale invariance;
- the observation-fibre postprocessing obstruction;
- position-only invisibility of orientation;
- equality transport and a commuting-frame transport theorem;
- the Sophie Germain polynomial identity;

- Euler’s identity through Mathlib’s theorem `Complex.exp_pi_mul_I`.

The Noether discussion is referenced standard mathematics but is not re-formalized in this small Lean file; a full formalization would require a declared variational framework far beyond the elementary kernel of this note.

A second point matters for the paper’s thesis. Mathlib deliberately totalizes inverse and division at zero in its `GroupWithZero` hierarchy. Thus, inside that formal interface,

$$0^{-1} = 0, \quad a/0 = 0.$$

The paper’s ordinary division contract is instead partial and explicitly requires $b \neq 0$. The Lean file encodes that contract as `safeDiv a b hb`, where `hb : b ≠ 0` is part of the input.

This is not a defect in Lean or Mathlib. It is a demonstration of formal scope:

A verifier proves the proposition expressed by the definitions supplied to it. It does not choose which informal semantic contract the author intended.

Astra 15A makes the same distinction between internal checking and external warrant: its scope section says that selecting a record exposes its warrant rather than manufacturing a verdict outside the record’s domain; its integrity section also notes that a file cannot authenticate itself merely by recomputing a hash stored inside itself.

Accordingly,

Lean-checked theorem $\neq$ unqualified semantic truth oracle.

The completed Lean Web run supplies a receipt for a formal derivation in the observed Lean 4.33.0 environment. The service did not expose its exact Mathlib commit; Appendix E records the separate, pinned reproduction target. The bridge from that formal proposition to an informal target remains a separately inspectable relation.

13. Verification ledger

The status below refers to the corrected source accepted in the captured run. Appendix B supplies the declaration names and axiom dependencies.

ID	Claim	Accepted scope / status
L1	$a + 0 = a, a - 0 = a$	Accepted over $\mathbb{Z}$
L2	$a - b = a + (-b)$	Accepted over $\mathbb{Z}$
L3	$a0 = 0$	Accepted over $\mathbb{Z}$
L4	$a(n + 1) = an + a$	Accepted over $\mathbb{N}$
L5	Finite Cartesian cardinality multiplies	Accepted for two- and three-factor Fin products
L6	Reciprocal involution	Accepted over $\mathbb{Q}$
L7	Uniqueness of the inverse restoring 1	Accepted over $\mathbb{Q}, b \neq 0$
L8	Division with an explicit nonzero denominator	<code>safeDiv</code> and its identity accepted over $\mathbb{Q}$
L9	Homogeneous swap squares to identity	Accepted on rational pairs
L10	Swap exchanges origin and infinity representatives	Accepted on rational pairs
L11	Swap preserves the projective cross-relation	Accepted on rational pairs
L12	Nonzero $(a, 0)$ has the infinity cross-relation	Accepted on rational pairs
L13	Common nonzero scale preserves the affine ratio	Accepted over $\mathbb{Q}$, with nonzero scale and denominator

ID	Claim	Accepted scope / status
L14	Postprocessing cannot split an observation fibre	Accepted for the declared maps
L15	A position-only registry hides orientation	Accepted for the declared state projection
L16	Equality transports through a function	Accepted for the declared function
L17	A commuting frame bridge transports identity	Accepted assuming commutation
L18	Sophie Germain polynomial identity	Accepted in a commutative ring
L19	Euler $e^{i\pi} + 1 = 0$	Accepted through Mathlib
L20	Noether first/second theorem discussion	Referenced; not formalized in this core

The encoded coordinate model uses rational pairs and their cross-multiplication relation. The core does not construct a projective quotient, prove the homeomorphism $\mathbb{RP}^1 \cong S^1$, or construct the particular bridges between all rungs. The bridge theorem takes commutation as an explicit premise.

Conclusion

The Dimensional Ladder is a way of displaying relations already built into standard mathematics.

The signed line gives an additive frame about a common origin. Natural-number multiplication calls that additive structure across an additional independent index, and Cartesian products make the representation exact. Division by a nonzero factor uses the unique reciprocal required to restore multiplicative identity. Projective completion then supplies an exact reciprocal frame-edge law:

$$[X : Z] \leftrightarrow [Z : X], \quad x \leftrightarrow \frac{1}{x}, \quad 0 \leftrightarrow \infty.$$

Projection supplies the complementary obstruction: variation that remains within one observation fibre cannot become displacement in the lower registry until a new independent relation is supplied.

Identity remains literal throughout. What changes from rung to rung is the formal description carrying it. Transport across a rung therefore requires an explicit bridge. Once that bridge is specified, preserved identity is ordinary mathematics; without it, shared typography does not establish shared meaning.

This is why the familiar rules need not be treated as arbitrary inscriptions that happen to work. Relative to their declared structures, many of them are forced by the requirement that the rest of the structure continue to satisfy its defining laws. The reciprocal is the elementary example. The projective flip is its exact dual-frame expression at the affine edge.

These features existed before this paper named their common pattern. **Lambda** is therefore a registry label for the recurrent relation, not an additional object inserted into the mathematics.

References

1. Lean Project, *Lean Language Reference*, release notes and proof-validation documentation, 2026. <https://lean-lang.org/doc/reference/latest/>
2. Lean Project, *Lean 4.33.0 Release Notes*, 10 August 2026. <https://lean-lang.org/doc/reference/latest/releases/v4.33.0/>
3. Mathlib, *Mathlib.Algebra.GroupWithZero.Defs*, current Mathlib 4 documentation. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/GroupWithZero/Defs.html
4. Mathlib, *Mathlib.Analysis.SpecialFunctions.Trigonometric.Basic*, including `Complex.exp_pi_mul_I`. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Analysis/SpecialFunctions/Trigonometric/Basic.html
5. Encyclopedia of Mathematics, *Noether theorem*, first and second theorem statements. https://encyclopediaofmath.org/index.php?title=Noether_theorem
6. Lu Semita, *Astra 15A — dimensional enlargement*, EmergenceByDesign, 2026, especially the exact-arithmetic orientation result and §§5–6 on scope and verification.

Source-boundary note

References 1–5 support the stated formal-system and standard mathematical infrastructure. Reference 6 supplies prior project scope discipline and an exact example in which metric data determine orientation only up to a global sign. None of these references is offered as authority for the Lambda label. The Lambda interpretation is the author’s proposed registry-level organization of the explicit results stated above.

Appendix A. Completed Lean verification receipt

Result: ACCEPTED. The public Lean Web service accepted the corrected V2 core and its audit commands. The captured run reports 30 theorem type checks, one definition type check, and 31 axiom reports. All evidence below comes from that completed run.

Receipt field	Observed value
Capture time (UTC)	2026-09-09 19:20:16.632
Service / project	Lean Web / <code>mathlib-stable</code>
Selected environment	Stable Release (v4.33.0 with mathlib, cslib)
Reported Lean version	4.33.0, also returned by <code>#eval Lean.versionString</code>
Theorems / definition checked	30 of 30 theorem declarations; <code>safeDiv</code>
Diagnostics	0 errors; 0 warnings; 64 informational messages
Axiom observations	31 reports; no <code>sorryAx</code> or nonstandard axiom in the reported closures
Submitted-source match	The source decoded from the captured editor URL matches the submitted UTF-8 bytes exactly.
Completion marker	<code>DIMENSIONAL_LADDER_V2_AUDIT_COMPLETE</code>

A.1 Corrections made before acceptance

The original V2 submission produced a syntax error at the scale-invariance theorem and an unnecessary-simplification warning at the reciprocal-chart proof. Two targeted edits resolved those diagnostics:

1. The bound variable λ used reserved Lean syntax. Renaming it to `scale`, and its hypothesis to `hscale`, removed the parse error; the corresponding expressions retain the same proposition and hypotheses.
2. The proof of `charts_are_reciprocal` now uses `exact (inv_div p.1 p.2).symm`, replacing the unnecessary `simp` `[uChart, xChart]` using `... step`.

No theorem was removed, weakened, or replaced by an assumption. The original and accepted sources, together with their exact diff, remain embedded in this PDF. An initial audit-harness trial also requested the unsupported display option `pp.width`; removing that option produced the final clean audit. Appendix C prints the final accepted submission.

A.2 What the acceptance covers

The acceptance covers the declarations and assumptions printed in Appendix C. The formal domains include integer arithmetic, finite natural-number products, rational reciprocals and coordinate pairs, generic observation and transport maps, commutative-ring polynomial algebra, and the stated complex-exponential identity. Section 13 gives the claim-to-scope mapping.

Across the 31 axiom reports, the union contains only `propext`, `Classical.choice`, and `Quot.sound`. Six theorem reports list no axioms. Appendix B gives the full mapping; Appendix D preserves the captured messages.

The service reported Lean 4.33.0 but did not expose its exact installed Mathlib commit. Local compilation stopped before elaboration because the runtime could not locate the Lean executable through its process path. No local kernel replay, independent kernel implementation, or independent human review ran. This receipt records browser-observed public-service output, with no service signature; it supplies execution evidence for the encoded core within the observed imported environment.

Appendix B. Complete axiom ledger

Every declaration below uses the `DimensionalLadder` namespace. **P** denotes `propext`; **C** denotes `Classical.choice`; **Q** denotes `Quot.sound`. “None” records an empty list from `#print axioms`. Every listed declaration passed its type check in the final run.

Declaration	Claim	Axioms
<code>add_zero_role</code>	L1	P
<code>sub_zero_role</code>	L1	P
<code>subtraction_as_add_inverse</code>	L2	P
<code>additive_reflection_involutive</code>	Related: L2	P
<code>mul_zero_role</code>	L3	P
<code>multiplication_calls_addition</code>	L4	None
<code>grid_card</code>	L5	P, C, Q
<code>box_card</code>	L5	P, C, Q
<code>reciprocal_involution</code>	L6	P, C, Q
<code>reciprocal_forced</code>	L7	P, C, Q
<code>safeDiv_eq_mul_inv</code>	L8	P, C, Q
<code>mathlib_div_zero_is_totalized</code>	Related: L8	P, C, Q
<code>flip_involutive</code>	L9	None
<code>flip_origin</code>	L10	P, Q
<code>flip_infinity</code>	L10	P, Q
<code>origin_valid</code>	Related: L10	P, C, Q
<code>infinity_valid</code>	Related: L10	P, C, Q
<code>flip_preserves_valid</code>	Related: L9	P, Q
<code>projEq_refl</code>	Related: L11	P, C, Q
<code>projEq_flip_iff</code>	L11	P, C, Q
<code>nonzero_over_zero_at_infinity</code>	L12	P, C, Q
<code>zero_zero_invalid</code>	Related: L12	P, Q
<code>charts_are_reciprocal</code>	Related: L13	P, C, Q
<code>ratio_scale_invariant</code>	L13	P, C, Q
<code>postprocess_cannot_split</code>	L14	None
<code>orientation_invisible_to_position</code>	L15	None
<code>equality_transports_through_function</code>	L16	None
<code>commuting_bridge_transports_identity</code>	L17	None
<code>sophie_germain</code>	L18	P, Q
<code>euler_identity</code>	L19	P, C, Q
<code>safeDiv</code>	L8	P, C, Q

`safeDiv` supplies the audited definition; the preceding 30 entries supply the theorem set. “Related” identifies supporting declarations beyond the shorter claim wording in Section 13.

Appendix C. Exact accepted Lean submission

The following listing reproduces `DimensionalLadder_WebAudit.lean`: the complete corrected mathematical core, followed by the type checks, axiom queries, version query, and completion marker used in the accepted run. Printed line numbers refer to this exact submission; visual wrapping does not add source lines. The PDF also embeds the original UTF-8 file for byte-exact extraction.

```

1 import Mathlib
2
3 /-!
4 # Expression of the Dimensional Ladder: Lean-checkable mathematical core
5
6 This file formalizes the narrow mathematical propositions used by the companion
7 paper. The label "Lambda" is not used as an axiom and is not required by any
8 proof below.
9
10 Pinned target: Lean 4.33.0 + Mathlib v4.33.0.
11 -/
12
13 namespace DimensionalLadder
14
15 /-! ## 1. Zero, sign, and primitive arithmetic -/
16
17 theorem add_zero_role (a : ℤ) : a + 0 = a := by simp
18
19 theorem sub_zero_role (a : ℤ) : a - 0 = a := by simp
20
21 theorem subtraction_as_add_inverse (a b : ℤ) : a - b = a + (-b) := by
22   simp [sub_eq_add_neg]
23
24 theorem additive_reflection_involutive (a : ℤ) : -(-a) = a := by simp
25
26 theorem mul_zero_role (a : ℤ) : a * 0 = 0 := by simp
27
28 /-! ## 2. Multiplication as repeated additive calling -/
29
30 theorem multiplication_calls_addition (a n : ℕ) :
31   a * (n + 1) = a * n + a := by
32   simp [Nat.succ_eq_add_one] using Nat.mul_succ a n
33
34 /- A finite `a × b` Cartesian grid has exactly `a*b` cells. -/
35 theorem grid_card (a b : ℕ) :
36   Fintype.card (Fin a × Fin b) = a * b := by
37   simp
38
39 /- Three independent finite enumeration coordinates multiply cardinalities. -/
40 theorem box_card (a b c : ℕ) :
41   Fintype.card (Fin a × Fin b × Fin c) = a * b * c := by
42   simp [mul_assoc]
43
44 /-! ## 3. Reciprocal necessity inside a field -/
45
46 theorem reciprocal_involution (x : ℚ) : (x⁻¹)⁻¹ = x := by
47   simp
48
49 /-
50 If `c` restores the multiplicative identity for a nonzero rational `b`, then
51 `c` is the reciprocal of `b`. This is the uniqueness claim used in the paper.
52 -/
53 theorem reciprocal_forced (b c : ℚ) (hb : b ≠ 0) (hbc : b * c = 1) :
54   c = b⁻¹ := by
55   calc
56     c = 1 * c := by simp
57     _ = (b⁻¹ * b) * c := by rw [inv_mul_cancel hb]
58     _ = b⁻¹ * (b * c) := by simp [mul_assoc]
59     _ = b⁻¹ := by rw [hbc]; simp

```

```

60
61 /-- The paper's intended division contract carries nonzeroness explicitly. -/
62 def safeDiv (a b : ℚ) (hb : b ≠ 0) : ℚ := a / b
63
64 theorem safeDiv_eq_mul_inv (a b : ℚ) (hb : b ≠ 0) :
65   safeDiv a b hb = a * b⁻¹ := by
66   simp [safeDiv, div_eq_mul_inv]
67
68 /-- Mathlib deliberately totalizes division at zero; this is not `safeDiv`. -/
69 theorem mathlib_div_zero_is_totalized (a : ℚ) : a / 0 = 0 := by
70   simp
71
72 /-! ## 4. Minimal homogeneous-coordinate model of the reciprocal frame -/
73
74 abbrev HCoord := ℚ × ℚ
75
76 def Valid (p : HCoord) : Prop := p.1 ≠ 0 ∨ p.2 ≠ 0
77
78 def flip (p : HCoord) : HCoord := (p.2, p.1)
79
80 def origin : HCoord := (0, 1)
81
82 def infinity : HCoord := (1, 0)
83
84 theorem flip_involutive (p : HCoord) : flip (flip p) = p := by
85   rcases p with (x, z)
86   rfl
87
88 theorem flip_origin : flip origin = infinity := by rfl
89
90 theorem flip_infinity : flip infinity = origin := by rfl
91
92 theorem origin_valid : Valid origin := by simp [Valid, origin]
93
94 theorem infinity_valid : Valid infinity := by simp [Valid, infinity]
95
96 theorem flip_preserves_valid (p : HCoord) : Valid (flip p) ↔ Valid p := by
97   rcases p with (x, z)
98   simp [Valid, flip, or_comm]
99
100 /-- Cross-multiplication relation underlying equality of projective ratios. -/
101 def projEq (p q : HCoord) : Prop := p.1 * q.2 = q.1 * p.2
102
103 theorem projEq_refl (p : HCoord) : projEq p p := by
104   simp [projEq]
105
106 theorem projEq_flip_iff (p q : HCoord) :
107   projEq (flip p) (flip q) ↔ projEq p q := by
108   rcases p with (x, z)
109   rcases q with (y, w)
110   constructor <=> intro h
111   · dsimp [projEq, flip] at h ⊢
112     simpa [mul_comm] using h.symm
113   · dsimp [projEq, flip] at h ⊢
114     simpa [mul_comm] using h.symm
115
116 /-- Every nonzero pair `(a,0)` is in the projective class represented by `(1,0)`. -/
117 theorem nonzero_over_zero_at_infinity (a : ℚ) (ha : a ≠ 0) :
118   Valid (a, 0) ∧ projEq (a, 0) infinity := by
119   constructor
120   · exact 0r.inl ha
121   · simp [projEq, infinity]
122
123 theorem zero_zero_invalid : ¬ Valid (0, 0) := by
124   simp [Valid]
125
126 /-! ## 5. Reciprocal affine charts and common-scale invariance -/

```

```

127
128 def xChart (p : HCoord) (_hz : p.2 ≠ 0) : ℚ := p.1 / p.2
129
130 def uChart (p : HCoord) (_hx : p.1 ≠ 0) : ℚ := p.2 / p.1
131
132 theorem charts_are_reciprocal (p : HCoord) (hx : p.1 ≠ 0) (hz : p.2 ≠ 0) :
133   uChart p hx = (xChart p hz)-1 := by
134     exact (inv_div p.1 p.2).symm
135
136 theorem ratio_scale_invariant
137   (x z scale : ℚ) (hz : z ≠ 0) (hscale : scale ≠ 0) :
138     (scale * x) / (scale * z) = x / z := by
139       field_simp [hz, hscale]
140
141 /-! ## 6. Observation fibres and invisible degrees of freedom -/
142
143 def Indistinguishable {X Y : Type} (obs : X → Y) (x₁ x₂ : X) : Prop :=
144   obs x₁ = obs x₂
145
146 theorem postprocess_cannot_split
147   {X Y Z : Type} (obs : X → Y) (g : Y → Z) {x₁ x₂ : X}
148   (h : Indistinguishable obs x₁ x₂) :
149     g (obs x₁) = g (obs x₂) := by
150       exact congrArg g h
151
152 structure State (P 0 : Type) where
153   pos : P
154   orient : 0
155
156 def observePosition {P 0 : Type} (s : State P 0) : P := s.pos
157
158 theorem orientation_invisible_to_position
159   {P 0 : Type} (p : P) (o₁ o₂ : 0) :
160     observePosition (State.mk p o₁) = observePosition (State.mk p o₂) := by
161       rfl
162
163 /-! ## 7. Literal identity and transport through a declared bridge -/
164
165 theorem equality_transports_through_function
166   {A B : Type} (F : A → B) {x y : A} (h : x = y) :
167     F x = F y := by
168       exact congrArg F h
169
170 /- A commuting semantic bridge between a source and successor frame. -/
171 def Commutes
172   {L D L' D' : Type}
173   (eval : L → D) (eval' : L' → D')
174   (translate : L → L') (carrier : D → D') : Prop :=
175   ∀ t, carrier (eval t) = eval' (translate t)
176
177 theorem commuting_bridge_transports_identity
178   {L D L' D' : Type}
179   (eval : L → D) (eval' : L' → D')
180   (translate : L → L') (carrier : D → D')
181   (hcomm : Commutes eval eval' translate carrier)
182   {s t : L} (hst : eval s = eval t) :
183     eval' (translate s) = eval' (translate t) := by
184       rw [← hcomm s, ← hcomm t, hst]
185
186 /-! ## 8. Exact identities at different representational rungs -/
187
188 theorem sophie_germain {R : Type} [CommRing R] (a b : R) :
189   a4 + 4 * b4 =
190     (a2 - 2*a*b + 2*b2) * (a2 + 2*a*b + 2*b2) := by
191     ring
192
193 /- Euler's distinguished-value identity, using Mathlib's established theorem. -/

```

```

194 theorem euler_identity :
195   Complex.exp ((Real.pi : ℂ) * Complex.I) + 1 = 0 := by
196   rw [Complex.exp_pi_mul_I]
197   ring
198
199 end DimensionalLadder
200
201 /-! Verification-only observations follow the mathematical source. -/
202
203
204 #check DimensionalLadder.add_zero_role
205 #print axioms DimensionalLadder.add_zero_role
206
207 #check DimensionalLadder.sub_zero_role
208 #print axioms DimensionalLadder.sub_zero_role
209
210 #check DimensionalLadder.subtraction_as_add_inverse
211 #print axioms DimensionalLadder.subtraction_as_add_inverse
212
213 #check DimensionalLadder.additive_reflection_involutive
214 #print axioms DimensionalLadder.additive_reflection_involutive
215
216 #check DimensionalLadder.mul_zero_role
217 #print axioms DimensionalLadder.mul_zero_role
218
219 #check DimensionalLadder.multiplication_calls_addition
220 #print axioms DimensionalLadder.multiplication_calls_addition
221
222 #check DimensionalLadder.grid_card
223 #print axioms DimensionalLadder.grid_card
224
225 #check DimensionalLadder.box_card
226 #print axioms DimensionalLadder.box_card
227
228 #check DimensionalLadder.reciprocal_involution
229 #print axioms DimensionalLadder.reciprocal_involution
230
231 #check DimensionalLadder.reciprocal_forced
232 #print axioms DimensionalLadder.reciprocal_forced
233
234 #check DimensionalLadder.safeDiv_eq_mul_inv
235 #print axioms DimensionalLadder.safeDiv_eq_mul_inv
236
237 #check DimensionalLadder.mathlib_div_zero_is_totalized
238 #print axioms DimensionalLadder.mathlib_div_zero_is_totalized
239
240 #check DimensionalLadder.flip_involutive
241 #print axioms DimensionalLadder.flip_involutive
242
243 #check DimensionalLadder.flip_origin
244 #print axioms DimensionalLadder.flip_origin
245
246 #check DimensionalLadder.flip_infinity
247 #print axioms DimensionalLadder.flip_infinity
248
249 #check DimensionalLadder.origin_valid
250 #print axioms DimensionalLadder.origin_valid
251
252 #check DimensionalLadder.infinity_valid
253 #print axioms DimensionalLadder.infinity_valid
254
255 #check DimensionalLadder.flip_preserves_valid
256 #print axioms DimensionalLadder.flip_preserves_valid
257
258 #check DimensionalLadder.projEq_refl
259 #print axioms DimensionalLadder.projEq_refl
260

```

```
261 #check DimensionalLadder.projEq_flip_iff
262 #print axioms DimensionalLadder.projEq_flip_iff
263
264 #check DimensionalLadder.nonzero_over_zero_at_infinity
265 #print axioms DimensionalLadder.nonzero_over_zero_at_infinity
266
267 #check DimensionalLadder.zero_zero_invalid
268 #print axioms DimensionalLadder.zero_zero_invalid
269
270 #check DimensionalLadder.charts_are_reciprocal
271 #print axioms DimensionalLadder.charts_are_reciprocal
272
273 #check DimensionalLadder.ratio_scale_invariant
274 #print axioms DimensionalLadder.ratio_scale_invariant
275
276 #check DimensionalLadder.postprocess_cannot_split
277 #print axioms DimensionalLadder.postprocess_cannot_split
278
279 #check DimensionalLadder.orientation_invisible_to_position
280 #print axioms DimensionalLadder.orientation_invisible_to_position
281
282 #check DimensionalLadder.equality_transports_through_function
283 #print axioms DimensionalLadder.equality_transports_through_function
284
285 #check DimensionalLadder.commuting_bridge_transports_identity
286 #print axioms DimensionalLadder.commuting_bridge_transports_identity
287
288 #check DimensionalLadder.sophie_germain
289 #print axioms DimensionalLadder.sophie_germain
290
291 #check DimensionalLadder.euler_identity
292 #print axioms DimensionalLadder.euler_identity
293
294 #check DimensionalLadder.safeDiv
295 #print axioms DimensionalLadder.safeDiv
296
297 #eval Lean.versionString
298 #eval "DIMENSIONAL_LADDER_V2_AUDIT_COMPLETE"
```

Appendix D. Complete captured run output

This transcript preserves the full captured Lean Web message text, including the editor's line references, all 31 type observations and 31 axiom reports, the version output, and the completion marker. The opening "No info found." describes the editor cursor position; the message panel below it contains the 64 informational messages. Appendix A records the final diagnostic counts. Visual line wrapping serves page layout only.

```

mathlib-stable.lean:299:0
No info found.
All Messages (64)
mathlib-stable.lean:204:0
DimensionalLadder.add_zero_role (a : ℤ) : a + 0 = a
mathlib-stable.lean:205:0
'DimensionalLadder.add_zero_role' depends on axioms: [propext]
mathlib-stable.lean:207:0
DimensionalLadder.sub_zero_role (a : ℤ) : a - 0 = a
mathlib-stable.lean:208:0
'DimensionalLadder.sub_zero_role' depends on axioms: [propext]
mathlib-stable.lean:210:0
DimensionalLadder.subtraction_as_add_inverse (a b : ℤ) : a - b = a + -b
mathlib-stable.lean:211:0
'DimensionalLadder.subtraction_as_add_inverse' depends on axioms: [propext]
mathlib-stable.lean:213:0
DimensionalLadder.additive_reflection_involutive (a : ℤ) : - -a = a
mathlib-stable.lean:214:0
'DimensionalLadder.additive_reflection_involutive' depends on axioms: [propext]
mathlib-stable.lean:216:0
DimensionalLadder.mul_zero_role (a : ℤ) : a * 0 = 0
mathlib-stable.lean:217:0
'DimensionalLadder.mul_zero_role' depends on axioms: [propext]
mathlib-stable.lean:219:0
DimensionalLadder.multiplication_calls_addition (a n : ℕ) : a * (n + 1) = a * n + a
mathlib-stable.lean:220:0
'DimensionalLadder.multiplication_calls_addition' does not depend on any axioms
mathlib-stable.lean:222:0
DimensionalLadder.grid_card (a b : ℕ) : Fintype.card (Fin a × Fin b) = a * b
mathlib-stable.lean:223:0
'DimensionalLadder.grid_card' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:225:0
DimensionalLadder.box_card (a b c : ℕ) : Fintype.card (Fin a × Fin b × Fin c) = a * b * c
mathlib-stable.lean:226:0
'DimensionalLadder.box_card' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:228:0
DimensionalLadder.reciprocal_involution (x : ℚ) : x-1-1 = x
mathlib-stable.lean:229:0
'DimensionalLadder.reciprocal_involution' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:231:0
DimensionalLadder.reciprocal_forced (b c : ℚ) (hb : b ≠ 0) (hbc : b * c = 1) : c = b-1
mathlib-stable.lean:232:0
'DimensionalLadder.reciprocal_forced' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:234:0
DimensionalLadder.safeDiv_eq_mul_inv (a b : ℚ) (hb : b ≠ 0) : DimensionalLadder.safeDiv a b hb = a * b-1
mathlib-stable.lean:235:0
'DimensionalLadder.safeDiv_eq_mul_inv' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:237:0
DimensionalLadder.mathlib_div_zero_is_totalized (a : ℚ) : a / 0 = 0
mathlib-stable.lean:238:0
'DimensionalLadder.mathlib_div_zero_is_totalized' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:240:0
DimensionalLadder.flip_involutive (p : DimensionalLadder.HCoord) : DimensionalLadder.flip (DimensionalLadder.flip p) = p
mathlib-stable.lean:241:0
'DimensionalLadder.flip_involutive' does not depend on any axioms
mathlib-stable.lean:243:0
DimensionalLadder.flip_origin : DimensionalLadder.flip DimensionalLadder.origin = DimensionalLadder.infinity
mathlib-stable.lean:244:0

```

```

'DimensionalLadder.flip_origin' depends on axioms: [propext, Quot.sound]
mathlib-stable.lean:246:0
DimensionalLadder.flip_infinity : DimensionalLadder.flip DimensionalLadder.infinity = DimensionalLadder.origin
mathlib-stable.lean:247:0
'DimensionalLadder.flip_infinity' depends on axioms: [propext, Quot.sound]
mathlib-stable.lean:249:0
DimensionalLadder.origin_valid : DimensionalLadder.Valid DimensionalLadder.origin
mathlib-stable.lean:250:0
'DimensionalLadder.origin_valid' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:252:0
DimensionalLadder.infinity_valid : DimensionalLadder.Valid DimensionalLadder.infinity
mathlib-stable.lean:253:0
'DimensionalLadder.infinity_valid' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:255:0
DimensionalLadder.flip_preserves_valid (p : DimensionalLadder.HCoord) :
  DimensionalLadder.Valid (DimensionalLadder.flip p) ↔ DimensionalLadder.Valid p
mathlib-stable.lean:256:0
'DimensionalLadder.flip_preserves_valid' depends on axioms: [propext, Quot.sound]
mathlib-stable.lean:258:0
DimensionalLadder.projEq_refl (p : DimensionalLadder.HCoord) : DimensionalLadder.projEq p p
mathlib-stable.lean:259:0
'DimensionalLadder.projEq_refl' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:261:0
DimensionalLadder.projEq_flip_iff (p q : DimensionalLadder.HCoord) :
  DimensionalLadder.projEq (DimensionalLadder.flip p) (DimensionalLadder.flip q) ↔ DimensionalLadder.projEq p q
mathlib-stable.lean:262:0
'DimensionalLadder.projEq_flip_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:264:0
DimensionalLadder.nonzero_over_zero_at_infinity (a : ℚ) (ha : a ≠ 0) :
  DimensionalLadder.Valid (a, 0) ∧ DimensionalLadder.projEq (a, 0) DimensionalLadder.infinity
mathlib-stable.lean:265:0
'DimensionalLadder.nonzero_over_zero_at_infinity' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:267:0
DimensionalLadder.zero_zero_invalid : ¬DimensionalLadder.Valid (0, 0)
mathlib-stable.lean:268:0
'DimensionalLadder.zero_zero_invalid' depends on axioms: [propext, Quot.sound]
mathlib-stable.lean:270:0
DimensionalLadder.charts_are_reciprocal (p : DimensionalLadder.HCoord) (hx : p.1 ≠ 0) (hz : p.2 ≠ 0) :
  DimensionalLadder.uChart p hx = (DimensionalLadder.xChart p hz)-1
mathlib-stable.lean:271:0
'DimensionalLadder.charts_are_reciprocal' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:273:0
DimensionalLadder.ratio_scale_invariant (x z scale : ℚ) (hz : z ≠ 0) (hscale : scale ≠ 0) :
  scale * x / (scale * z) = x / z
mathlib-stable.lean:274:0
'DimensionalLadder.ratio_scale_invariant' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:276:0
DimensionalLadder.postprocess_cannot_split {X Y Z : Type} (obs : X → Y) (g : Y → Z) {x₁ x₂ : X}
  (h : DimensionalLadder.Indistinguishable obs x₁ x₂) : g (obs x₁) = g (obs x₂)
mathlib-stable.lean:277:0
'DimensionalLadder.postprocess_cannot_split' does not depend on any axioms
mathlib-stable.lean:279:0
DimensionalLadder.orientation_invisible_to_position {P O : Type} (p : P) (o₁ o₂ : O) :
  DimensionalLadder.observePosition { pos := p, orient := o₁ } =
    DimensionalLadder.observePosition { pos := p, orient := o₂ }
mathlib-stable.lean:280:0
'DimensionalLadder.orientation_invisible_to_position' does not depend on any axioms
mathlib-stable.lean:282:0
DimensionalLadder.equality_transports_through_function {A B : Type} (F : A → B) {x y : A} (h : x = y) : F x = F y
mathlib-stable.lean:283:0
'DimensionalLadder.equality_transports_through_function' does not depend on any axioms
mathlib-stable.lean:285:0
DimensionalLadder.commuting_bridge_transports_identity {L D L' D' : Type} (eval : L → D) (eval' : L' → D')
  (translate : L → L') (carrier : D → D') (hcomm : DimensionalLadder.Commutates eval eval' translate carrier) {s t : L}
  (hst : eval s = eval t) : eval' (translate s) = eval' (translate t)
mathlib-stable.lean:286:0
'DimensionalLadder.commuting_bridge_transports_identity' does not depend on any axioms

```

```

mathlib-stable.lean:288:0
DimensionalLadder.sophie_germain {R : Type} [CommRing R] (a b : R) :
  a ^ 4 + 4 * b ^ 4 = (a ^ 2 - 2 * a * b + 2 * b ^ 2) * (a ^ 2 + 2 * a * b + 2 * b ^ 2)
mathlib-stable.lean:289:0
'DimensionalLadder.sophie_germain' depends on axioms: [propext, Quot.sound]
mathlib-stable.lean:291:0
DimensionalLadder.euler_identity : Complex.exp (↑Real.pi * Complex.I) + 1 = 0
mathlib-stable.lean:292:0
'DimensionalLadder.euler_identity' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:294:0
DimensionalLadder.safeDiv (a b : ℚ) (_hb : b ≠ 0) : ℚ
mathlib-stable.lean:295:0
'DimensionalLadder.safeDiv' depends on axioms: [propext, Classical.choice, Quot.sound]
mathlib-stable.lean:297:0
"4.33.0"
mathlib-stable.lean:298:0
"DIMENSIONAL_LADDER_V2_AUDIT_COMPLETE"
Restart File

```

Appendix E. Source identity and reproduction

E.1 Source hashes

The captured editor URLs decoded to the exact original and accepted source bytes. The SHA-256 values below identify the original core, corrected core, and full accepted submission.

Original V2 core

64bafddab811a60323e5b0615c9bfd82fc188332da5aeb2e22f0f4f6a2633526

Corrected accepted core

e8d91090465b62b8ac78ac90ca65426fa71db60d2e6e4bf9ae696620f24fa0cf

Exact accepted web-audit submission

11e8832ae7bacdd032b2c116f06d646b2f27a9fed4d62612802204f303171b0e

All five original V2 manifest entries matched: paper PDF, DOCX, Markdown, original Lean core, and verification-status file. The embedded JSON receipt retains the expected and observed checksums. These hashes identify bytes without independently authenticating the service.

E.2 Reproduce the formal check

Open the captured-source link below, or load Appendix C into the `mathlib-stable` project at [Lean Web](#). After processing finishes, inspect all messages for the declaration and axiom reports, Lean version, diagnostics, and completion marker.

[Open the exact captured source in Lean Web](#). The stable environment may change after the capture date; a later run supplies a new observation.

The separate local reproduction target uses Lean `leanprover/lean4:v4.33.0` and Mathlib commit:

db584cd6d46c92f209a44c0f1c829460d327499d

This recorded Mathlib v4.33.0 target supplies a reproduction pin; the captured web output did not establish its remote commit, and local reproduction remains unexecuted.

E.3 Evidence included inside this PDF

The visible appendices include the receipt, source, and output. This PDF also embeds the following exact files, accessible through a PDF reader's attachments panel.

Embedded file	Contents
<code>DimensionalLadder.lean</code>	Corrected 30-theorem mathematical core
<code>DimensionalLadder_WebAudit.lean</code>	Exact accepted source and audit commands
<code>LEAN_WEB_VERIFICATION_EVIDENCE.json</code>	Machine-readable receipt, source, URL, diagnostics and provenance
<code>LEAN_WEB_MESSAGES.txt</code>	Complete captured message text printed in Appendix D
<code>AXIOM_LEDGER.json</code>	Per-declaration dependencies printed in Appendix B
<code>source_corrections.diff</code>	Exact two-edit diff from the original core
<code>Original_V2_DimensionalLadder.lean</code>	Original uncorrected V2 formal source
<code>Original_V2_Paper.md</code>	Original V2 paper text and references
<code>lean_web_final.jpg</code>	Screenshot of the accepted Lean Web result
<code>REPRODUCE_LEAN_WEB.url</code>	Captured source-bearing Lean Web link

The embedded `EVIDENCE_SHA256SUMS.txt` records the attachment checksums. This edition preserves the V2 argument, updates verification status, and integrates the evidence appendices.